

การพัฒนาเกมโดยประยุกต์ใช้เทคนิค Finite State Machine ในการควบคุมตัวละครและปัญญาประดิษฐ์

Development of a Game using Finite State Machine Techniques for Character Control and Artificial Intelligence

ชื่อผู้จัดทำ

นายจิรายุ สงบวาจา

นางสาวอรไพลิน สอาดเอี่ยม

นางสาววนิดา คล้ายสกุล

โครงงานวิจัยระดับปริญญาตรีนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรวิทยาศาสตรบัณฑิต
สาขาวิชาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์และเทคโนโลยี
มหาวิทยาลัยเทคโนโลยีราชมงคลสุวรรณภูมิ พ.ศ. 2569

การพัฒนาเกมโดยประยุกต์ใช้เทคนิค Finite State Machine ในการควบคุมตัวละครและปัญญาประดิษฐ์

Development of a Game using Finite State Machine Techniques for Character Control and Artificial Intelligence

ชื่อผู้จัดทำ

นายจิรายุ สงบวาจา

นางสาวอรไพลิน สอาดเอี่ยม

นางสาววนิดา คล้ายสกุล

โครงงานวิจัยระดับปริญญาตรีนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรวิทยาศาสตรบัณฑิต
สาขาวิชาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์และเทคโนโลยี
มหาวิทยาลัยเทคโนโลยีราชมงคลสุวรรณภูมิ พ.ศ. 2569

หัวข้อโครงงานวิจัย	การพัฒนาเกมโดยประยุกต์ใช้เทคนิค Finite State Machine ในการควบคุมตัวละครและปัญญาประดิษฐ์ Development of a Game using Finite State Machine Techniques for Character Control and Artificial Intelligence
ชื่อนักศึกษา	นายจิรายุ สงบวาจา นางสาวอรไพลิน สอาดเอี่ยม นางสาววนิดา คล้ายสกุล
อาจารย์ที่ปรึกษา	อาจารย์ธนวัฒน์ ถาวรกูล ผศ.วราภรณ์ มั่นทุ่ง
ระดับการศึกษา	ปริญญาตรี
สาขาวิชา	วิทยาการคอมพิวเตอร์
ปีการศึกษา	2569

บทคัดย่อ

โครงงานวิจัยนี้มีวัตถุประสงค์เพื่อศึกษาและพัฒนาเกมคอมพิวเตอร์สองมิติแนวเอาชีวิตรอดและลอบเร้น (Survival Stealth Game) โดยมุ่งเน้นการประยุกต์ใช้เทคนิคเครื่องสถานะจำกัด (Finite State Machine: FSM) ในการควบคุมสถานะของตัวละครผู้เล่นและระบบปัญญาประดิษฐ์ (Artificial Intelligence: AI) ภายในเกม เพื่อให้การทำงานของระบบมีโครงสร้างที่ชัดเจน ลดความซับซ้อนของโปรแกรม และเพิ่มประสิทธิภาพในการควบคุมพฤติกรรมของตัวละคร การพัฒนาเกมใช้โปรแกรม Unity Game Engine และเขียนโปรแกรมด้วยภาษา C# โดยนำแนวคิด FSM มาใช้เป็นโครงสร้างหลักในการจัดการสถานะของตัวละครผู้เล่น ได้แก่ การยืน การเดิน การนั่งย่อ และการหมอบคลาน ซึ่งแต่ละสถานะมีผลต่อพารามิเตอร์ภายในเกม เช่น ความเร็วในการเคลื่อนที่ ขนาดพื้นที่การชน (Collider) และระดับความดังของเสียงแบบเรียลไทม์ นอกจากนี้ได้พัฒนาระบบปัญญาประดิษฐ์ของตัวละครศัตรูให้สามารถตรวจจับผู้เล่นผ่านระบบการมองเห็นด้วยเทคนิค Raycasting และตอบสนองต่อระดับเสียงที่ผู้เล่นสร้างขึ้น.โดยทำงานร่วมกับอัลกอริทึมค้นหาเส้นทางแบบ A* (A* Pathfinding) เพื่อให้ศัตรูสามารถเคลื่อนที่หลบหลีกสิ่งกีดขวางและไล่ล่าผู้เล่นได้อย่างสมจริง ผลการดำเนินงานพบว่า การประยุกต์ใช้เทคนิค Finite State Machine ช่วยให้สามารถจัดการสถานะของตัวละครและเงื่อนไขการทำงานของระบบ AI ได้อย่างเป็นระบบ ลดความซับซ้อนของโค้ด และเพิ่มความยืดหยุ่นในการพัฒนา ซอฟต์แวร์ต้นแบบที่พัฒนาขึ้นสามารถตอบสนองต่อคำสั่งของผู้เล่นได้อย่างถูกต้อง และตัวละครศัตรูมีพฤติกรรมการไล่ล่าที่สอดคล้อง

กับตรรกะที่กำหนดไว้ บรรลุตามวัตถุประสงค์ของโครงงานในการประยุกต์ใช้ความรู้ด้านวิทยาการคอมพิวเตอร์เพื่อพัฒนาสื่อบันเทิงเชิงโต้ตอบ

กิตติกรรมประกาศ

โครงงานวิจัยฉบับนี้สำเร็จลุล่วงไปได้ด้วยดีเนื่องด้วยความกรุณาจาก อาจารย์ธนวัฒน์ ถาวรกูล และ ผศ.วราภรณ์ มั่นทุ่ง อาจารย์ที่ปรึกษาโครงงาน ที่ได้ให้คำปรึกษา แนะนำแนวทาง และแก้ไขข้อบกพร่องต่าง ๆ ในการพัฒนาเกมสยองขวัญ 2 มิติเชิงจริยธรรมด้วยภาษา C# ตลอดจนถ่ายทอดความรู้และเทคนิคต่าง ๆ ในการใช้โปรแกรม Unity จนโครงงานเล่มนี้เสร็จสมบูรณ์

ขอกราบขอบพระคุณบิดา มารดา และครอบครัว ที่ให้การสนับสนุนทั้งในด้านกำลังใจและทุนทรัพย์ในการศึกษาเล่าเรียนเสมอมา รวมถึงขอขอบคุณเพื่อนนักศึกษาทุกท่านที่ให้ความช่วยเหลือในการทดสอบระบบเกมและให้คำแนะนำที่เป็นประโยชน์ต่อการพัฒนาโครงงานในครั้งนี้

คุณค่าและประโยชน์ของโครงงานวิจัยฉบับนี้ ผู้จัดทำขอมอบเป็นเครื่องบูชาพระคุณบิดา มารดา ครูบาอาจารย์ และผู้มีพระคุณทุกท่านที่มีส่วนช่วยให้ผู้จัดทำประสบความสำเร็จในการศึกษา

ชื่อผู้จัดทำ

จิรายุ สงบวาจา

อรไพลิน สอาดเอี่ยม

วนิดา คล้ายสกุล

บทที่ 1

บทนำ

1.1 ความเป็นมาและความสำคัญของปัญหา

การพัฒนาเกมคอมพิวเตอร์ในปัจจุบันมีความซับซ้อนทางด้านวิศวกรรมซอฟต์แวร์มากขึ้น โดยเฉพาะในเกมแนวลอบเร้น (Stealth Survival) หัวใจสำคัญไม่ได้อยู่ที่ความสวยงามเพียงอย่างเดียว แต่อยู่ที่ "การออกแบบโครงสร้างโค้ด (Software Architecture)" เพื่อควบคุมสถานะของตัวละคร (Player State Management) ที่มีความละเอียดซับซ้อน เช่น การเปลี่ยนอิริยาบถจากการยืน เป็นการนั่งย่อ หรือการหมอบคลาน ซึ่งการกระทำเหล่านี้ต้องส่งผลต่อค่าพารามิเตอร์ต่างๆ ทางฟิสิกส์และการตรวจจับภายในเกมแบบเรียลไทม์

นอกจากนี้ การสร้างศัตรูคอมพิวเตอร์ (AI Agent) ให้มีความฉลาดและตอบสนองต่อพฤติกรรมของผู้เล่นอย่างสมจริง จำเป็นต้องประยุกต์ใช้อัลกอริทึมขั้นสูง เช่น Finite State Machine (FSM) ในการจำแนกสถานะ และการคำนวณทางคณิตศาสตร์เวกเตอร์ (Vector Mathematics) เพื่อตรวจสอบการมองเห็น (Line of Sight) และการได้ยินเสียงที่แปรผันตามท่าทางของผู้เล่น

ผู้จัดทำจึงมีแนวคิดพัฒนาโครงงาน "การพัฒนาเกมโดยประยุกต์ใช้เทคนิค Finite State Machine ในการควบคุมตัวละครและปัญญาประดิษฐ์" โดยใช้ Unity Game Engine และภาษา C# โครงงานนี้จะมุ่งเน้นการเขียนโปรแกรมเพื่อจัดการกลไกการเคลื่อนไหวที่ซับซ้อน (Advanced Movement Mechanics) และระบบ AI ที่ไล่ล่าผู้เล่นอย่างเป็นระบบ เพื่อเป็นกรณีศึกษาการนำความรู้วิทยาการคอมพิวเตอร์มาประยุกต์ใช้ในการสร้างซอฟต์แวร์ที่มีการโต้ตอบสูง

1.2 วัตถุประสงค์ของโครงงาน

1.2.1 เพื่อพัฒนาเกมคอมพิวเตอร์ 2 มิติ ด้วยภาษา C# โดยเน้นการวางโครงสร้างโค้ดแบบเชิงวัตถุ (Object-Oriented Programming)

1.2.2 เพื่อประยุกต์ใช้โครงสร้างข้อมูลแบบ Finite State Machine (FSM) ในการจัดการสถานะการเคลื่อนไหวของตัวละครผู้เล่น (Movement States

1.2.3 เพื่อพัฒนาอัลกอริทึมปัญญาประดิษฐ์ (AI) สำหรับศัตรูที่สามารถตอบสนองต่อระดับเสียงและขนาดพื้นที่การมองเห็น (Hitbox) ที่เปลี่ยนแปลงไปตามสถานะของผู้เล่นได้

1.2.4 เพื่อสร้างระบบการเล่น (Gameplay Loop) ที่สมบูรณ์ ทั้งการเก็บไอเทม การแก้ไขปริศนา และการเปลี่ยนฉากเมื่อผ่านเงื่อนไข

1.2.5 เพื่อประเมินความพึงพอใจของผู้เล่นต่อเกมที่พัฒนาขึ้น เพื่อนำไปปรับปรุงและพัฒนาระบบให้มีประสิทธิภาพยิ่งขึ้น

1.3 ขอบเขตของโครงงาน

1.3.1 ส่วนพัฒนาระบบควบคุมตัวละคร (Player Controller System)

พัฒนาระบบควบคุมตัวละครผู้เล่นให้สามารถเคลื่อนไหวได้หลากหลายรูปแบบ และส่งผลต่อการเล่นในเชิงกลยุทธ์ เช่น การลดเสียงเพื่อหลบศัตรู

1.3.1.1 State Management

พัฒนาระบบสถานะของตัวละครโดยใช้ Finite State Machine (FSM) เพื่อควบคุมการเปลี่ยนสถานะระหว่าง Idle, Walk, Crouch และ Prone อย่างเป็นลำดับ

1.3.1.2 Dynamic Collider Adjustment

พัฒนาระบบปรับขนาด Hitbox ตามท่าทางของตัวละครแบบเรียลไทม์ เพื่อรองรับการเคลื่อนที่ในพื้นที่จำกัด เช่น การหมอบลอดช่องแคบ

1.3.1.3 Variable Parameters

กำหนดค่าความเร็วและระดับเสียงในแต่ละสถานะ เพื่อใช้เป็นข้อมูลในการตรวจจับของศัตรู

1.3.2 ส่วนพัฒนาระบบปัญญาประดิษฐ์ของศัตรู (Enemy AI System)

พัฒนาระบบ AI ที่สามารถตอบสนองต่อพฤติกรรมของผู้เล่น โดยเน้นการตรวจจับและการตัดสินใจแบบเรียลไทม์

1.3.2.1 Behavior System (FSM)

กำหนดสถานะของศัตรู เช่น Patrol, Investigate, Chaseและ Attack เพื่อควบคุมพฤติกรรม

1.3.2.2 Sensory System

พัฒนาระบบการมองเห็นด้วย Raycasting และระบบการได้ยินโดยใช้การคำนวณระยะทางร่วมกับระดับเสียงของผู้เล่น

1.3.2.3 Pathfinding

ใช้ Navigation Mesh และ A* Algorithm เพื่อให้ศัตรูสามารถเคลื่อนที่หลบสิ่งกีดขวางและติดตามผู้เล่นได้

1.3.3 ส่วนพัฒนาระบบเกม (Game Logic System)

พัฒนากลไกหลักของเกมเพื่อกำหนดเป้าหมายและลำดับการเล่น

1.3.3.1 Inventory และ Mission Logic

ผู้เล่นต้องเก็บไอเทมให้ครบจำนวนที่กำหนด (เช่น 5 ชิ้น) เพื่อปลดล็อกทางออก

1.3.3.2 Scene Management

รองรับการเปลี่ยนฉากและการเก็บข้อมูลความคืบหน้าของเกม

1.3.4 ส่วนออกแบบส่วนติดต่อผู้ใช้งาน (UI/UX)

เป็นการออกแบบส่วนติดต่อผู้ใช้เพื่อแสดงข้อมูลสำคัญ เช่น สถานะตัวละคร ระดับเสียง และจำนวนไอเทม เพื่อช่วยให้ผู้เล่นสามารถวางแผนการเล่นและตัดสินใจได้อย่างมีประสิทธิภาพ

1.3.5 ขอบเขตด้านเนื้อหาเกม (Game Content Scope)

1.3.5.1 เกมที่พัฒนาเป็นเกมคอมพิวเตอร์แบบ 2 มิติ แนวลอบเร้น (Stealth Gameโดยผู้เล่นต้องหลบหลีกศัตรูและวางแผนการเคลื่อนที่

1.3.5.2 ภายในเกมมีฉากการเล่นอย่างน้อย 1–2 ฉาก ซึ่งออกแบบให้มีสิ่งกีดขวางและพื้นที่สำหรับการหลบซ่อน

1.3.5.3 มีตัวละครศัตรูอย่างน้อย 1 ประเภท ที่สามารถลาดตระเวน ตรวจจับเสียง และไล่ล่าผู้เล่นได้

1.3.5.4 มีระบบการเก็บไอเทม โดยผู้เล่นต้องรวบรวมไอเทมให้ครบตามจำนวนที่กำหนดเพื่อผ่านด่าน

1.3.5.5 มีเงื่อนไขการชนะ (Win Condition) เมื่อผู้เล่นเก็บไอเทมครบและไปถึงทางออกของฉาก

1.3.5.6 มีเงื่อนไขการแพ้ (Lose Condition) เมื่อผู้เล่นถูกศัตรูตรวจจับและพลังชีวิหมด

1.3.6 ขอบเขตด้านเทคโนโลยี (Technology Scope)

1.3.6.1 ใช้ Unity Game Engine ในการพัฒนาเกม

1.3.6.2 ใช้ภาษา C# ในการเขียนโปรแกรมควบคุมระบบต่าง ๆ ภายในเกม

1.3.6.3 ใช้แนวคิดการเขียนโปรแกรมเชิงวัตถุ (Object-Oriented Programming OOP) ในการออกแบบโครงสร้างระบบ

1.3.6.4 ใช้เทคนิค Finite State Machine (FSM) ในการควบคุมพฤติกรรมของตัละคผู้เล่นและศัตรู

1.3.6.5 ใช้ Raycasting ในการพัฒนาระบบการมองเห็นของศัตรู และใช้การคำนวณระยะทางสำหรับระบบตรวจจับเสียง

1.3.6.6 ใช้ Navigation Mesh ร่วมกับอัลกอริทึม A* (A-Star Algorithm) ในการค้นหาเส้นทางของศัตรู

1.4 เครื่องมือที่เกี่ยวข้องกับโครงงาน

1.4.1 Unity Game Engine เครื่องมือหลักในการพัฒนา

1.4.2 ภาษา C# สำหรับเขียนสคริปต์ควบคุมการทำงาน

1.4.3 Visual Studio: IDE สำหรับเขียนโค้ดและ Debug

1.4.4 โปรแกรมสร้างกราฟิก 2D: สำหรับสร้าง Sprite ตัวละครและฉาก

1.5 ตารางการดำเนินโครงงาน

การดำเนินโครงงานวิจัย	ระยะเวลาการดำเนินโครงงานวิจัย																							
	พ.ย.		ธ.ค.		ม.ค.		ก.พ.		มี.ค.		เม.ย.		พ.ค.		มิ.ย.		ก.ค.		ส.ค.		ก.ย.	
1. ศึกษาปัญหาและความต้องการ																						
2. กำหนดขอบเขต																						
3. ศึกษาการใช้งาน Unity																						
4. ออกแบบตัวละคร																						
5. ออกแบบระบบ																						
6. พัฒนาระบบ																						
7. ติดตั้งโปรแกรมและทดสอบ																						
8. ปรับปรุงและแก้ไขระบบ																						
10. ทำเอกสารงานวิจัย																						
11. พบอาจารย์ที่ปรึกษา																						

ระยะเวลาคาดหวัง

ระยะเวลาทำงานจริง

1.6 ประโยชน์ที่คาดว่าจะได้รับ

1.6.1 ได้รับความรู้ความเข้าใจเชิงลึกในการเขียนโปรแกรมจัดการสถานะ (State Management) ซึ่งเป็นพื้นฐานสำคัญของการพัฒนาซอฟต์แวร์

1.6.2 สามารถประยุกต์ใช้คณิตศาสตร์และอัลกอริทึม (Physics & Algorithms) ในการสร้างระบบการตรวจจับ (Detection System) ภายในเกมได้

1.6.3 ได้ต้นแบบซอฟต์แวร์เกมที่มีระบบการควบคุมตัวละคร (Character Controller) และ AI ที่ทำงานสัมพันธ์กันอย่างเป็นระบบ

1.6.4 มีทักษะในการออกแบบ UX/UI ที่สื่อสารข้อมูลเชิงเทคนิค (เช่น ระดับเสียง) ให้ผู้ใช้งานเข้าใจได้ง่าย

1.6.5 ทำให้ทราบระดับความพึงพอใจของผู้เล่น และสามารถนำผลที่ได้ไปใช้ในการปรับปรุงและพัฒนาระบบเกมให้มีประสิทธิภาพและตอบสนองต่อผู้ใช้งานได้ดียิ่งขึ้น

บทที่ 2

ทฤษฎีและงานวิจัยที่เกี่ยวข้อง

ในการพัฒนาเกมคอมพิวเตอร์ 2 มิติ แนว Side-scrolling Hide-and-Seek Horror Game ด้วย C# ผู้จัดทำได้ศึกษาหลักการทฤษฎีและเทคโนโลยีที่ เกี่ยวข้อง เพื่อใช้เป็นแนวทางในการดำเนินงานวิจัย ประกอบไปด้วยหัวข้อหลัก ๆ ดังต่อไปนี้

2.1 แนวคิดเกี่ยวกับการพัฒนาเกม (Game Development)

2.2 การพัฒนาเกม 2 มิติ

2.3 เกม Side-scrolling game หรือ Side-view game

2.4 เกมประเภท Hide-and-Seek Horror Game

2.5 แนวคิดเกี่ยวกับ Finite State Machine (FSM)

2.6 การพัฒนาปัญญาประดิษฐ์ในเกม

2.7 ภาษาและเครื่องมือที่ใช้พัฒนา

2.8 อัลกอริทึมการค้นหาเส้นทาง (Pathfinding Algorithm)

2.9 การออกแบบ ux ui ให้เหมาะสม

2.10 งานวิจัยที่เกี่ยวข้อง

2.1 แนวคิดเกี่ยวกับการพัฒนาเกม (Game Development)

การพัฒนาเกม (Game Development) เป็นกระบวนการสร้างซอฟต์แวร์ เชิงโต้ตอบที่ผสานองค์ประกอบด้านการออกแบบ ศิลปะ และวิศวกรรมซอฟต์แวร์เข้าด้วย กัน โดยทั่วไปกระบวนการพัฒนาเกมประกอบด้วยขั้นตอนการวิเคราะห์ความต้องการ (Requirement Analysis) การออกแบบระบบ (System Design) การพัฒนา (Implementation) การทดสอบ (Testing) และการประเมินผล (Evaluation) กระบวนการสร้างเกม (Game Development) คือ กระบวนการในมุมมอง ของการสร้าง การออกแบบ โดยใช้ประกอบในมุมมองสองมิติ โดยการออกประกอบจาก เกม X (แนวนอน) และ แนวนอน Y (แนวตั้ง) ซึ่งแตกต่างจากเกม 3 มิติที่สามารถแสดง ตัวละครจากเกม ในการรวมกันสร้างสรรค์เกมในการออกแบบ เกม 2 มิติจะดำเนินการใน พื้นที่ หรือสิ่งต่างๆ ในการออกแบบ ทั้งนี้ เกม 2 มิติที่มีสามารถแสดงสถานการณ์สำหรับผู้ เล่นในเนื้อหาหรืออาจจะใช้ในการออกแบบแนวเกมสนุก ที่ตัวละครจะสร้างสรรค์กลยุทธ์ และสามารถสร้างความประทับใจให้ผู้เล่นได้ องค์ประกอบสำคัญของเกม ได้แก่ ระบบกลไกการเล่น (Game Mechanics) ระบบกราฟิกและแอนิเมชัน ระบบเสียง ระบบปัญญาประดิษฐ์ (AI System) และส่วน ติดต่อผู้ใช้ (User Interface) ในเกมที่มีศัตรูหรือ NPC ระบบ AI มีบทบาทสำคัญใน

การสร้างความท้าทายและความสมจริง โดยเฉพาะเกมแนวสยองขวัญ (Horror Game) ที่พฤติกรรมของศัตรูต้องตอบสนองต่อการกระทำของผู้เล่นอย่างมีเหตุผล

2.2 การพัฒนาเกม 2 มิติ

การพัฒนาเกม 2 มิติ (2D Game Development) เป็นการสร้างเกมที่ใช้ พิกัดสองมิติในการแสดงผล ประกอบด้วยแกน X (แนวนอน) และแกน Y (แนวตั้ง) ซึ่ง แตกต่างจากเกม 3 มิติที่มีความลึก (Depth) เพิ่มเติม เกม 2 มิติมีความได้เปรียบใน ด้านความง่ายต่อการพัฒนา สุนทรียศาสตร์ที่มีเอกลักษณ์ได้ ใช้ทรัพยากรการประมวลผลต่ำกว่า และสามารถสร้าง นักพัฒนาจะต้องออกแบบและควบคุมสภาพแวดล้อม โดยระบบดำเนินการเส้นทาง (Platformer) ร่วมลักษณะการทำงานในสามารถทำงานได้ดีในเกม 2 มิติ โดยแตก ต่างจากเกม 3 มิติในด้านความซับซ้อน ในการพัฒนาเกม 2 มิติ เครื่องมือที่นิยมใช้คือUnity Game Engine ซึ่งรองรับการพัฒนาเกม 2 มิติได้อย่างครบถ้วน ทั้งระบบ Sprite, Tilemaps, Physics 2D และ Animator สำหรับการสร้างแอนิเมชั่นตัว ละคร ส่วนสำคัญที่สำคัญในการพัฒนาเกม 2 มิติ คือ การออกแบบระดับ (Level Design) ซึ่งเป็นการเลือกรูปแบบและทรัพยากรในการพัฒนาเกม รวมถึงการออกแบบ Tilemap เพื่อสร้างฉากที่มีความสวยงามและน่าสนใจ นอกจากนี้ระบบ Collision Detection ก็มีความสำคัญในการตรวจจับการชนระหว่างตัวละครกับสิ่งแวดล้อม ภายในเกม

2.3 เกม Side-scrolling game หรือ Side-view game

เกม Side-scrolling game หรือ Side-view game คือเกมที่มุมกล้อง ถูกกำหนดให้มองจากด้านข้างของตัวละคร ทำให้ผู้เล่นเห็นตัวละครในแนวตั้งและสามารถ เคลื่อนที่ไปทางซ้ายหรือขวาตามแกน X เป็นหลัก มุมมองแบบนี้เป็นที่นิยมอย่างกว้างขวางในเกมยุคแรก ๆ และยังคงได้รับความนิยมอย่างต่อเนื่องในเกมยุคปัจจุบัน เนื่องจาก สามารถสร้างประสบการณ์การเล่นที่ชัดเจนและเข้าใจง่าย คุณลักษณะสำคัญของเกม Side-scrolling ได้แก่ การเคลื่อนที่ของตัวละครใน แนวแกน X เป็นหลัก กล้องจะเลื่อนตาม (Follow Camera) ไปพร้อมกับตัวละคร การออกแบบฉากในแนวนอนที่มีความต่อเนื่อง รวมถึงการใช้ระบบชั้น (Layering) เพื่อสร้างความลึกในภาพแบบ 2 มิติ เช่น ฉากหลังเลื่อนช้ากว่าฉากหน้า (Parallax Scrolling) ซึ่งช่วยเพิ่มความสมจริงให้กับเกม ในการพัฒนาเกม Side-scrolling Horror Game นั้น มุมมองด้านข้างช่วย สร้างบรรยากาศความหวาดกลัวได้อย่างมีประสิทธิภาพ เนื่องจากผู้เล่นจะมองเห็นสิ่งที่อยู่ ข้างหน้าและข้างหลังได้อย่างชัดเจน แต่ไม่สามารถมองเห็นสิ่งที่อยู่ด้านบนหรือด้านล่างได้ ครบถ้วน ทำให้เกิดความตื่นเต้นและความไม่แน่นอนขณะเล่น

2.4 เกมประเภท Hide-and-Seek Horror Game

เกมประเภท Hide-and-Seek Horror Game คือเกมสยองขวัญที่มีแกน หลักของการเล่น (Core Gameplay) อยู่ที่การหลบซ่อนตัวจากศัตรูหรือผู้ไล่ล่า (Pursuer) ผู้เล่นจะต้องใช้ไหวพริบและความระมัดระวังในการหลบหลีกศัตรู รวบรวม ไอเทมที่กำหนด และหาทางออกจากด่านให้สำเร็จ โดยไม่ถูกศัตรูจับได้ เกมประเภทนี้มีชื่อเสียงเช่น Slender: The Eight Pages, Outlast, และ Amnesia: The Dark Descent องค์ประกอบสำคัญของ Hide-and-Seek Horror Game ได้แก่ ระบบ ตรวจจับของศัตรู (Enemy Detection System) ซึ่งประกอบด้วยการมองเห็น (Line of Sight) และการได้ยินเสียง (Sound Detection) ผู้เล่นจะต้องควบคุม ระดับเสียงที่เกิดจากการเคลื่อนไหว เช่น การเดินธรรมดาจะมีเสียงดังกว่าการนั่งย่อหรือหมอบคลาน รวมถึงการใช้สภาพแวดล้อม เช่น การซ่อนตัวในตู้หรือใต้โต๊ะ เพื่อหลบเลี่ยง การตรวจจับในเกม 2 มิติแนว Side-scrolling Horror Game การออกแบบระบบ Al ของศัตรูมีความสำคัญอย่างยิ่ง ศัตรูจะต้องมีพฤติกรรมที่สมเหตุสมผลและน่าเชื่อถือ โดย อาศัยเทคนิค Finite State Machine (FSM) ในการจัดการสถานะพฤติกรรมต่าง ๆ เช่น การลาดตระเวน การตรวจสอบเสียง และการไล่ล่า รวมถึงระบบ Raycasting สำหรับตรวจสอบการมองเห็น และอัลกอริทึม A* สำหรับการนำทางในฉาก

2.5 แนวคิดเกี่ยวกับ Finite State Machine (FSM)

Finite State Machine (FSM) หรือเครื่องสถานะจำกัด เป็นแบบจำลอง และการ การคำนวณที่ใช้กำหนดพฤติกรรมของระบบผ่านชุดของสถานะ (States) เปลี่ยนผ่านสถานะ (Transitions) ตามเงื่อนไขที่กำหนด โดยในช่วงเวลาใดเวลาหนึ่ง ระบบจะอยู่ได้เพียงสถานะเดียวเท่านั้น FSM เป็นที่นิยมอย่างกว้างขวางในการพัฒนาเกม เนื่องจากเข้าใจง่าย ใช้ทรัพยากรประมวลผลต่ำ และสามารถปรับแก้พฤติกรรมได้โดยไม่กระทบโครงสร้างทั้งหมด

องค์ประกอบหลักของ FSM ได้แก่ สถานะเริ่มต้น (Initial State) สถานะการ ทำงาน (Operational States) เหตุการณ์หรือเงื่อนไข (Event/Condition) การเปลี่ยนสถานะ (Transition) และการกระทำภายในสถานะ (State Action) ใน โครงงานนี้ FSM ถูกนำมาใช้ควบคุมทั้งตัวละครผู้เล่นและ Al ของศัตรู สำหรับตัวละครผู้เล่นใน Hide-and-Seek Horror Game สถานะที่ใช้ใน FSM ได้แก่ การยืนเดิน (Walk) ความเร็วปกติและรัศมีเสียงกว้าง การนั่งย่อ(Crouch) ความเร็วลดลงและรัศมีเสียงปานกลาง และการหมอบคลาน (Prone) ความเร็วต่ำที่สุดและรัศมีเสียงเงียบที่สุด แต่ละสถานะส่งผลต่อขนาด Collider และระดับ Noise Radius ซึ่งเป็นปัจจัยหลักในการถูกตรวจจับโดย A สำหรับ Al ของศัตรู (Ghost/Killer) สถานะที่ใช้ใน FSM ได้แก่ Patrol State (ลาดตระเวนตาม Waypoints), Investigate State (ตรวจสอบจุดที่ ได้ยินเสียง), Chase State (ไล่ล่าเมื่อเห็นผู้เล่นใน Vision Cone) และ Attack State (โจมตีเมื่อระยะห่าง ≤ 2 pixel) นอกจากนี้ FSM ยังกล่าวถึง Hierarchical Finite State Machine (HFSM) ซึ่งเป็น

การจัดลำดับชั้นของสถานะเพื่อลดความ ซับซ้อนของ FSM แบบดั้งเดิม ทำให้สามารถบริหารจัดการพฤติกรรมที่ซับซ้อนได้ดียิ่งขึ้น

2.6 การพัฒนาปัญญาประดิษฐ์ในเกม

Game Al (Game Artificial Intelligence) หมายถึง ระบบที่ควบคุม พฤติกรรมของตัวละครในเกมให้สามารถโต้ตอบกับผู้เล่นได้อย่างสมเหตุสมผล เป้าหมาย หลักของ Game Al ไม่ใช่การเรียนรู้ชั้นสูงแบบ Machine Learning แต่เป็นการ สร้าง "ภาพลวงตาของความฉลาด" เพื่อเพิ่มความสนุกและความท้าทายในการเล่น เทคนิคที่นิยมใช้ใน Game Al ได้แก่ Rule-based System, Finite State Machine (FSM), Behavior Tree และ Pathfinding Algorithm ในเกม 2 มิติแนว Hide-and-Seek Horror Game ระบบ Al ของศัตรูจำเป็นต้อง ผสมผสานเทคนิคหลายอย่างเข้าด้วยกัน ดังนี้

2.6.1 Finite State Machine (FSM) สำหรับศัตรู ใช้ FSM ในการจัดการสถานะพฤติกรรมของศัตรู (Ghost/Killer Al) โดยแบ่ง ออกเป็นสถานะ Patrol, Investigate, Chase และ Attack โดยแต่ละสถานะจะเปลี่ยนไปตามเงื่อนไขที่ตรวจจับได้จากสภาพแวดล้อม เช่น ระยะสายตา และระดับเสียงของผู้เล่น

2.6.2 Sensory System (ระบบการรับรู้) พัฒนาระบบรับรู้ของ Al โดยใช้ Raycasting ในการจำลองสายตา (Line of Sight / Vision Cone) เพื่อตรวจสอบว่าผู้เล่นอยู่ในแนวสายตาของศัตรูหรือไม่ และใช้การคำนวณระยะทาง (Distance Check) เปรียบเทียบกับ Noise Radius ที่ผู้เล่นสร้างขึ้นตามท่าทางการเคลื่อนไหว

2.6.3 การค้นหาเส้นทาง ใช้ระบบ Navigation ร่วมกับ A* Algorithm ให้ Al สามารถเดินไล่ล่าผู้เล่น หลบสิ่งกีดขวาง และเคลื่อนที่ไปยังจุดที่ได้ยินเสียงล่าสุดได้อย่างสมจริงภายในฉาก เกม 2 มิติแนว Side-scrolling

2.7 ภาษาและเครื่องมือที่ใช้พัฒนา

2.7.1 Visual Studio Code

เป็นโปรแกรมแก้ไขโค้ด (Code Editor) ที่พัฒนาโดย Microsoft มีความ ยืดหยุ่นสูง รองรับภาษาโปรแกรมต่างๆ ด้วยส่วนขยาย (Extensions) ที่หลาก หลาย สามารถเชื่อมต่อกับภาษา C# และ Unity Game Engine ได้อย่าง สะดวก มีระบบ IntelliSense ช่วยแนะนำโค้ดและแก้ไขข้อผิดพลาดได้อย่างมี ประสิทธิภาพ

2.7.2 เอ็นจิ้นเกม Unity

เป็นแอปพลิเคชันสำหรับการพัฒนาเกม มีความยืดหยุ่นสูง รองรับการพัฒนาเกมทั้ง 2 มิติ และ 3 มิติ โดยสามารถส่งออกเกมไปยัง Windows, Android และ iOS ได้อย่างสะดวก Unity มีเครื่องมือสำคัญดังนี้

2.7.2.1 Scene Editor ใช้ในการออกแบบฉากและจัดวางวัตถุใน เกม รวมถึงการสร้าง Tilemap สำหรับเกม 2 มิติแนว Side-scrolling

2.7.2.2 Animator ใช้ควบคุมและแสดงผลแอนิเมชันของตัวละครใน สถานะต่าง ๆ เช่น Walk, Crouch, Prone

2.7.2.3 Physics Engine 2D จัดการระบบฟิสิกส์ การชน (Collision) และการเคลื่อนที่ของวัตถุในเกม 2 มิติ

2.7.3 ภาษาโปรแกรม C#

เป็นภาษาหลักที่ใช้ในการพัฒนาเกมใน Unity มีโครงสร้างเชิงวัตถุ (OOP) ที่ ชัดเจน รองรับการสร้างระบบที่ซับซ้อน เช่น ระบบ FSM ระบบ AI ระบบ Inventory และการจัดการสถานะตัวละคร โดยในโครงงานนี้ใช้ C# เขียน สคริปต์ควบคุมการทำงานทั้งหมด

2.7.3.1 Game Logic Scripting สำหรับเขียนตรรกะการทำงาน ของระบบต่าง ๆ เช่น การเปลี่ยนสถานะ FSM และการตรวจจับ

2.7.3.2 Event Handling ใช้จัดการเหตุการณ์ต่าง ๆ เช่น Input ของผู้เล่น การชน และการเปลี่ยนฉาก

2.7.3.3 Data Management ใช้จัดการข้อมูลไอเทม สถานะผู้เล่น และการส่งต่อข้อมูลข้ามฉาก (Data Persistence)

2.7.4 JSON (JavaScript Object Notation)

เป็นรูปแบบไฟล์ข้อมูลที่มีโครงสร้างแบบคู่ระหว่างชื่อ (Key) และค่า (Value) ซึ่งอ่านและแก้ไขได้ง่ายในรูปแบบ Human-Readable Format สามารถใช้ ร่วมกับภาษา C# ใน Unity เพื่อบันทึกและโหลดข้อมูลเกม เช่น ข้อมูลตัวละคร และความคืบหน้าในการเล่น

2.8 อัลกอริทึมการค้นหาเส้นทาง (Pathfinding Algorithm)

ระบบค้นหาเส้นทาง (Pathfinding) มีบทบาทสำคัญในการทำให้ Al ของศัตรู สามารถเคลื่อนที่ภายในฉากเกม 2 มิติ Side-scrolling ได้อย่างมีประสิทธิภาพและ หลบหลีกสิ่งกีดขวางต่าง ๆ ได้อย่างสมจริง A* Algorithm (A-Star) เป็นอัลกอริทึมที่นิยมใช้ในเกม เนื่องจากสามารถ คำนวณเส้นทางที่เหมาะสมที่สุด โดยพิจารณาทั้งค่าใช้จ่ายจริงจากจุดเริ่มต้น (g-cost) และค่าประมาณระยะทางไปยังเป้าหมาย (h-cost) โดย f(n) = g(n) + h(n) ทำให้ ได้เส้นทางที่สั้นที่สุดในเวลาอันรวดเร็ว การทำงานร่วมกันระหว่าง FSM และ A* ใน Hide-and-Seek Horror Game มีลักษณะดังนี้ เมื่อ FSM ของศัตรูเปลี่ยนเป็นสถานะ Investigate ระบบจะ เรียกใช้ A* เพื่อนำทางไปยังตำแหน่งที่ได้ยินเสียงล่าสุด และเมื่อเปลี่ยนเป็นสถานะ Chase ระบบจะคำนวณเส้นทางไปยังตำแหน่งปัจจุบันของผู้เล่นแบบเรียลไทม์ โครงสร้างนี้ช่วยให้ศัตรูมีพฤติกรรมการไล่ล่าที่สมจริงและน่าตื่นเต้นยิ่งขึ้น

2.9 การออกแบบ UX/UI ให้เหมาะสม

การออกแบบส่วนติดต่อผู้ใช้ (User Interface: UI) และประสบการณ์ผู้ใช้ (User Experience: UX) เป็นองค์ประกอบสำคัญในการพัฒนาเกมที่ช่วยให้ผู้เล่น สามารถโต้ตอบกับระบบได้อย่างสะดวกและเข้าใจง่าย ในเกม 2 มิติ Side-scrolling Hide-and-Seek Horror Game การออกแบบ UI ที่ดีจะต้องไม่บดบังทัศนวิสัย ของผู้เล่น แต่ยังคงสื่อสารข้อมูลสำคัญได้อย่างชัดเจน องค์ประกอบ UI ที่สำคัญในเกมประเภทนี้ ได้แก่ ตัวบ่งชี้ท่าทาง (Stance Indicator) ที่แสดงสถานะปัจจุบันของผู้เล่นว่ากำลังยืน นั่งย่อ หรือหมอบคลาน มิเตอร์ เสียง (Noise Meter) ที่แสดงระดับความดังของเสียงที่ผู้เล่นสร้างขึ้นแบบเรียลไทม์ พลังชีวิต (HP Bar) และตัวนับไอเทม (Item Counter) เพื่อให้ผู้เล่นทราบความคืบ หน้าของภารกิจ หลักการออกแบบ UX ที่ดีในเกม Horror คือการใช้สีและเสียงเพื่อสร้าง บรรยากาศความกลัว โดยไม่ทำให้ UI ดูไม่สอดคล้องกับโทนของเกม นอกจากนี้ควร ออกแบบ HUD (Head-Up Display) ที่กระชับ สื่อความหมายได้ชัดเจน และตอบ สนองต่อสถานการณ์ในเกมได้อย่างทันท่วงที และน่าตื่นเต้น เพื่อสร้างประสบการณ์การเล่นที่น่าจดจำและน่าตื่นเต้น

2.10 งานวิจัยที่เกี่ยวข้อง

2.10.1 Finite State Machine in Game Development

งานวิจัยนี้ศึกษาแนวคิดและหลักการทำงานของ Finite State Machine (FSM) ในการพัฒนาระบบปัญญาประดิษฐ์ (AI) สำหรับควบคุมพฤติกรรมของตัวละครที่ไม่ใช่ผู้เล่น (Non-Player Characters: NPC) ภายในเกม โดยอธิบายว่า FSM เป็นแบบจำลองที่ประกอบด้วย “สถานะ (State)” และ “การเปลี่ยนสถานะ (Transition)” ซึ่งจะเปลี่ยนไปตามเงื่อนไขที่กำหนดไว้ล่วงหน้า การประยุกต์ใช้ FSM ในเกมช่วยให้สามารถกำหนดพฤติกรรมของตัวละคร เช่น การเดิน การไล่ล่า การโจมตี การป้องกัน หรือการหลบหนี ได้อย่างเป็นระบบ ลดความซับซ้อนของโค้ด และเพิ่มความชัดเจนในการออกแบบระบบ นอกจากนี้ งานวิจัยยังกล่าวถึง Hierarchical Finite State Machine (HFSM) ซึ่งเป็นการจัดลำดับชั้นของสถานะเพื่อลดความซับซ้อนของ FSM แบบดั้งเดิม ทำให้สามารถบริหารจัดการพฤติกรรมที่ซับซ้อนได้ดียิ่งขึ้น ผลการศึกษาสรุปว่า FSM เป็นเทคนิคที่มีประสิทธิภาพ เข้าใจง่าย และยังคงได้รับความนิยมในการพัฒนาเกมสมัยใหม่

2.10.2 Development of Two Dimension (2D) Game Engine with Finite State Machine (FSM) Based Artificial Intelligence (AI) Subsystem

งานวิจัยนี้มุ่งพัฒนาเกมเอนจินแบบ 2 มิติ ที่มีระบบ AI ภายในโดยใช้แนวคิด FSM เป็นแกนหลักของการควบคุมพฤติกรรมตัวละคร พัฒนาด้วยเทคโนโลยีเว็บ เช่น JavaScript และ WebGL 2.0 มีการเปรียบเทียบประสิทธิภาพระหว่างการพัฒนา AI แบบทั่วไป (Naive Approach) กับการใช้ FSM เป็นโครงสร้างหลัก พบว่า การใช้ FSM สามารถรักษาค่าเฟรมเรต (Frames Per Second: FPS) ได้ที่

ประมาณ 60 FPS ขณะที่วิธีทั่วไปทำให้เฟรมเรต ลดลงเหลือประมาณ 50 FPS แสดงให้เห็นว่า FSM ช่วยเพิ่มประสิทธิภาพของระบบ ลดความกำกวมของตรรกะการทำงาน และช่วยให้การจัดการพฤติกรรมของตัวละครเป็นระบบมากขึ้น

2.10.3 An Analysis of Finite State Machine Based Enemy Artificial Intelligence in Kirby: Nightmare in Dream Land

บทความนี้ศึกษาการประยุกต์ใช้เทคนิค **Finite State Machine (FSM)** ในการออกแบบปัญญาประดิษฐ์ของศัตรูในเกม Kirby: Nightmare in Dream Land โดยมุ่งวิเคราะห์ว่าโครงสร้างแบบกำหนดสถานะสามารถควบคุมพฤติกรรมศัตรูและส่งผลต่อระดับความยากของเกมได้อย่างไร ผู้วิจัยอธิบายว่า FSM เป็นโมเดลที่แบ่งพฤติกรรมออกเป็นสถานะต่าง ๆ เช่น หยุดนิ่ง (Idle) เดินลาดตระเวน (Patrol) โจมตี (Attack) ถูกโจมตี (Hit) และฟื้นตัว (Recover) และกำหนดเงื่อนไขการเปลี่ยนสถานะอย่างชัดเจนตามเหตุการณ์ที่เกิดขึ้นในเกม จากการทดลองสังเกตพฤติกรรมศัตรูหลายครั้ง พบว่าสถานะการโจมตีมีสัดส่วนเวลามากที่สุด ขณะที่สถานะหยุดนิ่งและเดินช่วยสร้างจังหวะพักให้ผู้เล่น ทำให้เกมมีความสมดุล ไม่ยากหรือไม่ง่ายเกินไป นอกจากนี้ ผู้วิจัยยังทดลองปรับค่าระยะเวลาหน่วงก่อนการโจมตี (Attack Cooldown) และพบว่ายิ่งศัตรูโจมตีถี่ขึ้น ระดับความยากของเกมก็เพิ่มสูงขึ้นอย่างชัดเจน แสดงให้เห็นว่า FSM สามารถปรับความท้าทายของเกมได้โดยการแก้ไขพารามิเตอร์เพียงบางค่า โดยไม่ต้องปรับโครงสร้าง AI ทั้งระบบ โดยสรุป งานวิจัยชี้ให้เห็นว่า FSM เป็นเทคนิคที่มีความเสถียร ใช้ทรัพยากรประมวลผลต่ำ ควบคุมและปรับแต่งได้ง่าย และยังคงมีความเหมาะสมสำหรับการพัฒนา AI ในเกมแพลตฟอร์ม 2 มิติ แม้จะมีเทคนิค AI ที่ซับซ้อนกว่าพัฒนาขึ้นในปัจจุบันก็ตาม

2.10.4 การออกแบบเกมดิจิทัล

งานวิจัยนี้นำเสนอแนวคิดและกระบวนการออกแบบเกมดิจิทัล โดยกล่าวถึงหลักการออกแบบจากหลากหลายแนวทาง เช่น การออกแบบจากนวัตกรรม ความคิดสร้างสรรค์ เรื่องราว ความต้องการของผู้เล่น และข้อจำกัดด้านเทคโนโลยี นอกจากนี้ยังกล่าวถึงประเภทของเกม เช่น เกม 2 มิติ และ 3 มิติ รวมถึงการแบ่งตามลักษณะการเล่น เช่น เกมต่อสู้ เกมยิง เกมผจญภัย และเกมการศึกษา พร้อมทั้งเสนอขั้นตอนการพัฒนาเกมอย่างเป็นระบบ ได้แก่ การวางแผน การจัดทำเอกสารออกแบบเกม (Game Design Document: GDD) การพัฒนาโปรแกรม การสร้างวัตถุในเกม การทดสอบ และการเผยแพร่ ซึ่งเป็นแนวทางสำคัญในการพัฒนาเกมให้ประสบความสำเร็จ

2.10.5 การพัฒนาเกมเพื่อการเรียนรู้การเขียนโปรแกรมภาษา C# ด้วยเกมการผจญภัยของอาเรียกับเวทย์มนต์แห่ง C#

งานวิจัยนี้มีวัตถุประสงค์เพื่อพัฒนาเกมสำหรับเสริมการเรียนรู้การเขียนโปรแกรมภาษา C# โดยใช้โปรแกรม Unity ในการพัฒนาเกมแนวผจญภัย (Adventure Game) และใช้แนวคิด Game-Based Learning เพื่อให้ผู้เรียนเรียนรู้ผ่านการเล่นเกม ผลการประเมินโดยผู้เชี่ยวชาญจำนวน 5 คน

พบว่าคุณภาพของเกมอยู่ในระดับดีมาก (ค่าเฉลี่ย 4.71) และเมื่อนำไปทดลองกับกลุ่มตัวอย่างจำนวน 20 คน พบว่าคะแนนหลังเรียนเพิ่มขึ้นร้อยละ 40.92 แสดงให้เห็นว่าเกมสามารถช่วยเพิ่มความเข้าใจในการเขียนโปรแกรม และส่งเสริมการคิดอย่างเป็นระบบได้อย่างมีประสิทธิภาพ

2.10.6 A Systematic Review and Analysis of Intelligence-Based Pathfinding Algorithms in the Field of Video Games

งานวิจัยนี้นำเสนอการทบทวนและวิเคราะห์อัลกอริทึมการค้นหาเส้นทาง (Pathfinding Algorithm) ที่ใช้ปัญญาประดิษฐ์ในเกมวิดีโอ โดยจำแนกอัลกอริทึมออกเป็น 3 กลุ่มหลัก ได้แก่ กลุ่ม Uninformed Search เช่น BFS และ Dijkstra's กลุ่ม Informed Search เช่น A* และ Greedy Best First และกลุ่ม Metaheuristic เช่น Genetic Algorithm และ Ant Colony Optimization ผู้วิจัยได้ทดสอบอัลกอริทึมบน Grid 2 มิติขนาด 4×4, 8×8, 16×16 และ 32×32 โดยใช้ Unity Game Engine เป็นสภาพแวดล้อมในการทดสอบ วัดผลด้วยเวลาประมวลผล (Execution Time) จำนวนบล็อกที่ผ่าน และความยาวเส้นทางที่สั้นที่สุด ผลการทดลองพบว่า A* Algorithm มีความสมดุลระหว่างความเร็วและความแม่นยำสูงที่สุด โดยใช้เวลาประมวลผลน้อยกว่า Dijkstra's และ BFS ในทุกขนาด Grid ทั้งที่มีและไม่มีสิ่งกีดขวาง และยังรับประกันเส้นทางที่สั้นที่สุด ซึ่งต่างจาก Greedy Best First ที่แม้จะเร็วที่สุดแต่ไม่ให้เส้นทางที่เหมาะสมที่สุดเสมอไป งานวิจัยนี้ยืนยันว่า A* เป็นอัลกอริทึมที่เหมาะสมที่สุดสำหรับการนำทาง NPC ในเกม 2 มิติ ซึ่งสอดคล้องกับการออกแบบระบบ AI ของศัตรูในโครงงานนี้

2.10.7 Improving Mobile Game Performance with Basic Optimization Techniques in Unity

งานวิจัยนี้ศึกษาผลกระทบของเทคนิค Optimization พื้นฐานที่มีต่อประสิทธิภาพของเกมบนอุปกรณ์มือถือ Android โดยพัฒนาเกม Shooter 3 มิติด้วย Unity และ Blender จำนวน 2 เวอร์ชัน ได้แก่ เวอร์ชันที่ใช้เทคนิค Optimization และเวอร์ชันที่ไม่ใช้ โดยเทคนิคที่นำมาทดสอบประกอบด้วย Low Poly Model, Merge Vertices, Textures and Materials, Occlusion Culling และ Object Pooling ตัวชี้วัดประสิทธิภาพที่ใช้คือค่า Frames Per Second (FPS), Batches และจำนวน Triangles/Polygons ซึ่งสะท้อนการใช้งาน CPU, GPU และหน่วยความจำตามลำดับ ผลการทดสอบบนอุปกรณ์มือถือจริงพบว่าเวอร์ชันที่ใช้เทคนิค Optimization รักษาค่า FPS ได้ในช่วง 50–60 FPS ตลอดการเล่น ขณะที่เวอร์ชันที่ไม่ผ่านการ Optimize มีค่า FPS ตกลงเหลือเพียง 20–30 FPS เมื่อมีศัตรูจำนวนมากบนหน้าจอ ทำให้เกมเกิดอาการกระตุก งานวิจัยนี้ยืนยันว่าการเลือกใช้เทคนิค Optimization ที่เหมาะสมมีผลโดยตรงต่อความลื่นไหลของเกม ซึ่งสอดคล้องกับโครงงานนี้ที่ต้องการให้เกม Side-scrolling Horror ทำงานได้อย่างมีประสิทธิภาพบน Unity

บทที่ 3
วิธีการดำเนินงาน

การพัฒนาเกมคอมพิวเตอร์ 2 มิติแนว Side-scrolling Hide-and-Seek Horror Game ในโครงงานนี้ดำเนินการด้วย Unity Game Engine และภาษา C# โดยประยุกต์ใช้เทคนิค Finite State Machine หรือ FSM ในการควบคุมสถานะของตัวละครผู้เล่นและพฤติกรรมของศัตรู ระบบถูกแบ่งเป็นโมดูลเพื่อให้ส่วนควบคุมผู้เล่น ระบบศัตรู ระบบไอเท็ม ระบบฉาก ระบบเสียง และส่วนติดต่อผู้ใช้งานทำงานร่วมกันได้อย่างเป็นระบบและสามารถปรับปรุงต่อได้

เกมประกอบด้วย 3 ด่านซึ่งมีระดับความยากและค่าพารามิเตอร์แตกต่างกัน ผู้เล่นต้องสำรวจพื้นที่ เก็บกุญแจให้ครบ หลีกเลี่ยงศัตรู และเดินทางไปยังประตูทางออก โดยสามารถใช้ไฟฉาย จุดซ่อนตัว และก้อนหินสร้างเสียงล่อศัตรูเพื่อช่วยในการหลบหนี ขั้นตอนการดำเนินงานประกอบด้วยการกำหนดปัญหา การรวบรวมข้อมูล การวิเคราะห์ระบบ การออกแบบระบบ การพัฒนา การทดสอบ และการบำรุงรักษา

3.1 การกำหนดปัญหา

3.1.1 ปัญหาด้านการควบคุมตัวละครผู้เล่น

ตัวละครผู้เล่นต้องรองรับการเดิน การวิ่ง การกระโดด การไต่บันได การซ่อนตัว การใช้ไอเท็ม การได้รับความเสียหาย และการเปลี่ยนฉาก การทำงานเหล่านี้อาจเกิดขึ้นต่อเนื่องหรือขัดแย้งกัน หากไม่มีระบบควบคุมสถานะที่ชัดเจนอาจทำให้ตัวละครรับคำสั่งในช่วงที่ไม่ควรควบคุมได้ โครงงานจึงใช้ FSM กำหนดสถานะ Normal, Hidden, Frozen, Hurt, UsingItem, Transitioning และ Dead เพื่อควบคุมสิทธิ์การรับคำสั่งและพฤติกรรมในแต่ละช่วง

3.1.2 ปัญหาด้านพฤติกรรมของศัตรู

ศัตรูต้องตอบสนองต่อผู้เล่นได้หลายลักษณะ ทั้งการลาดตระเวน การหันหาเสียง การตรวจสอบพื้นที่ การค้นหา การไล่ล่า และการโจมตี หากไม่มีลำดับสถานะที่ชัดเจน ศัตรูอาจเปลี่ยนพฤติกรรมอย่างไม่สมเหตุสมผลหรือไม่สามารถกลับไปลาดตระเวนได้หลังจากสูญเสียเป้าหมาย จึงใช้ Enemy FSM ควบคุมลำดับการตัดสินใจจากข้อมูลการมองเห็น การได้ยิน ระดับความสงสัย และตำแหน่งล่าสุดของผู้เล่น

3.1.3 ปัญหาด้านระบบตรวจจับ

ระบบตรวจจับต้องพิจารณาระยะทาง มุมมอง สิ่งกีดขวาง การเปิดไฟฉาย และเสียงที่เกิดจากกิจกรรมต่าง ๆ ของผู้เล่น การวิ่งมีความสำคัญมากกว่าการเดิน ขณะที่ก้อนหินและกับดักกระจกสร้างเสียงจากตำแหน่งอื่นเพื่อเบี่ยงเบนความสนใจของศัตรู ระบบจึงจำแนกประเภทเสียงเป็น Footstep, Sprint, Rock, Glass และ Environmental ก่อนประเมินความดัง ระยะห่าง และอายุของเสียง

3.1.4 ปัญหาด้านตรรกะและเป้าหมายของเกม

ผู้เล่นต้องเก็บกุญแจตามจำนวนที่กำหนดก่อนเปิดประตูทางออก ระบบต้องตรวจสอบจำนวนกุญแจ สถานะประตู ระยะการโต้ตอบ และอันตรายรอบตัวผู้เล่น การปลดล็อกต้องกดปุ่มโต้ตอบค้างไว้และยกเลิกได้เมื่อผู้เล่นปล่อยปุ่ม เดินออกจากระยะ ถูกโจมตี หรือมีศัตรูอยู่ใกล้ เพื่อให้เป้าหมายของเกมชัดเจนและมีความท้าทาย

3.1.5 ปัญหาด้านส่วนติดต่อผู้ใช้งาน

ผู้เล่นต้องรับรู้พลังชีวิต แบตเตอรี่ไฟฉาย พลังงานสำหรับวิ่ง จำนวนก้อนหิน จำนวนกุญแจ ข้อความโต้ตอบ และสถานการณ์อันตรายแบบเรียลไทม์ โดยข้อมูลต้องไม่บดบังพื้นที่เล่น ระบบจึงแสดง HIDDEN เมื่อซ่อนตัว แสดง DANGER เมื่อถูกไล่ล่า และใช้เสียงหัวใจร่วมกับขอบจอสีแดงเมื่อศัตรูอยู่ใกล้

3.1.6 ปัญหาด้านการออกแบบและปรับสมดุลด่าน

แต่ละด่านต้องมีบรรยากาศและระดับความยากต่างกัน ค่าเกี่ยวกับผู้เล่น ศัตรู และไฟฉายจึงต้องปรับได้โดยไม่แก้โค้ดหลัก โครงงานใช้ LevelConfig แยกค่าของแต่ละด่าน และออกแบบ Level 3 เป็นป่ามืดที่มีหมอก ต้นไม้หลายชั้น ทางเดินต่างระดับ กับดักเสียง และตำแหน่งกุญแจที่เปลี่ยนแปลงได้

3.2 การรวบรวมข้อมูล

3.2.1 ข้อมูลด้านการพัฒนาเกม

ผู้พัฒนาได้ศึกษาการพัฒนาเกม 2 มิติด้วย Unity ระบบ Sprite และ Tilemap ระบบ Physics 2D การรับข้อมูลด้วย Unity Input System การจัดการฉาก การสร้าง UI และการใช้ ScriptableObject สำหรับจัดเก็บค่าตั้งต้นของด่าน
นอกจากนี้ยังศึกษาการเขียนโปรแกรมเชิงวัตถุเพื่อแยกความรับผิดชอบของแต่ละระบบให้สามารถพัฒนาและทดสอบได้ง่าย

3.2.2 ข้อมูลด้าน Finite State Machine

ศึกษาหลักการ FSM ซึ่งแบ่งการทำงานออกเป็นสถานะและกำหนดเงื่อนไขการเปลี่ยนสถานะอย่างชัดเจน หลักการดังกล่าวถูกนำมาใช้กับตัวละครผู้เล่น ศัตรู ประตูทางออก คลังไอเท็ม และระบบความตึงเครียด เพื่อป้องกันการทำงานซ้อนกันและช่วยให้ตรวจสอบพฤติกรรมได้เป็นลำดับ

3.2.3 ข้อมูลด้านเกมแนวซ่อนตัวและสยองขวัญ

ศึกษากลไกการมองเห็นและการได้ยินของศัตรู การใช้เสียงเป็นความเสี่ยงและเครื่องมือล่อ การออกแบบจุดซ่อน การจำกัดทรัพยากรไฟฉาย และการใช้เสียงดนตรี เสียงหัวใจ หมอก แสง และฉากหลังหลายชั้นเพื่อสร้างความกดดันแก่ผู้เล่น

3.3 การวิเคราะห์ระบบ

3.3.1 การวิเคราะห์ระบบควบคุมผู้เล่น

PlayerController รับคำสั่งจาก Unity Input System และควบคุมการเคลื่อนที่แนวนอน การวิ่ง การใช้ Stamina การกระโดด การไต่บันได การโต้ตอบ การเปิดไฟฉาย การซ่อนตัว และการหยุดเกม การวิ่งใช้พลังงานซึ่งลดลงระหว่างใช้งานและฟื้นกลับหลังจากหยุดพัก ส่วนระบบกระโดดมีการตรวจสอบพื้นและ Coyote Time
เพื่อช่วยให้การควบคุมตอบสนองเป็นธรรมชาติ

คำสั่ง	แป้นพิมพ์	ผลการทำงาน
เคลื่อนที่	W A S D หรือปุ่มลูกศร	เดินและกำหนดทิศทาง
วิ่ง	Left Shift	เพิ่มความเร็ว ใช้ Stamina และสร้างเสียงมากขึ้น
กระโดด	Space	กระโดดเมื่ออยู่บนพื้นหรือในช่วง Coyote Time
โต้ตอบ	E	เก็บไอเท็ม ใช้วัตถุ หรือกดค้างเพื่อปลดล็อกประตู
ไฟฉาย	F	เปิดหรือปิดไฟฉายและใช้พลังงานแบตเตอรี่
ซ่อนตัว	H	เข้าและออกจากจุดซ่อน
คลังไอเท็ม	C	เปิดหรือปิดหน้าต่างคลังไอเท็ม
ก้อนหิน	Q	ขว้างก้อนหินเพื่อสร้างเสียงล่อศัตรู
หยุดเกม	Escape	เปิดหรือปิด Pause Menu

3.3.2 การวิเคราะห์ Player FSM

สถานะ	หน้าที่
Normal	สถานะปกติที่รับคำสั่งเดิน วิ่ง กระโดด และโต้ตอบ
Hidden	ผู้เล่นอยู่ในจุดซ่อนและถูกจำกัดการเคลื่อนไหว
Frozen	หยุดการเคลื่อนไหวชั่วคราวและต้องทำตามเงื่อนไขเพื่อหลุดออก
Hurt	สถานะตอบสนองเมื่อได้รับความเสียหาย
UsingItem	ล็อกคำสั่งบางส่วนระหว่างเปิดคลังหรือใช้ไอเท็ม
Transitioning	ล็อกการควบคุมระหว่างการเปลี่ยนด่าน

สถานะ	หน้าที่
Dead	หยุดการควบคุมเมื่อพลังชีวิตหมด

3.3.3 การวิเคราะห์ระบบปัญญาประดิษฐ์ของศัตรู

KillerAI ประมวลผลแนวสายตาด้วย Raycasting และประมวลผลเสียงจากคิวเหตุการณ์ โดยให้ค่าน้ำหนักตามประเภทเสียง ความดัง ระยะห่าง และเวลาที่เสียงเกิดขึ้น
ศัตรูบันทึกตำแหน่งล่าสุดของผู้เล่นหรือเสียงไว้ชั่วคราว เพื่อนำไปใช้ในสถานะตรวจสอบและค้นหา
การเปิดไฟฉายเพิ่มระยะที่ศัตรูสามารถตรวจพบผู้เล่นด้วยสายตา
ส่วนผู้เล่นที่อยู่ในจุดซ่อนจะไม่ถูกตรวจจับตามเงื่อนไขปกติ

สถานะ	พฤติกรรม
Patrol	เดินลาดตระเวนตามจุดที่กำหนด
Alert	หันไปยังทิศทางของเหตุการณ์ผิดปกติ
Listening	หยุดเพื่อพิจารณาทิศทางและความสำคัญของเสียง
InvestigateSound	เคลื่อนที่ไปยังต้นกำเนิดเสียง
ConfirmSound	ตรวจสอบบริเวณต้นกำเนิดเสียง
Confused	ลังเลหลังพบการใช้ก้อนหินล่อซ้ำบริเวณเดิม
Investigate	ไปยังตำแหน่งล่าสุดของผู้เล่น
Search	ค้นหาบริเวณรอบตำแหน่งล่าสุด
Chase	ไล่ล่าผู้เล่นที่ถูกตรวจพบ
Attack	โจมตีผู้เล่นเมื่ออยู่ในระยะ
AttackRecover	หยุดชั่วคราวหลังการโจมตี
ReturnToPatrol	กลับไปยังเส้นทางลาดตระเวน
Scream	กรีดร้องเพื่อทำให้ผู้เล่น Stun หรือ Frozen ตามการตั้งค่า

3.3.4 การวิเคราะห์ระบบไอเท็มและภารกิจ

InventoryManager ใช้โครงสร้าง List
เก็บไอเท็มที่ผู้เล่นรวบรวมและแยกกุญแจออกจากไอเท็มทั่วไป ระบบส่งเหตุการณ์ไปยัง UI

เมื่อจำนวนกุญแจเปลี่ยนแปลง และตรวจสอบว่าผู้เล่นมีกุญแจครบตามค่า RequiredKeyCount หรือไม่ คลังไอเท็มเปิดด้วยปุ่ม C แสดงไอเท็มตามช่องที่กำหนด และอนุญาตให้ใช้ไอเท็มที่รองรับการใช้งาน

3.3.5 การวิเคราะห์ข้อมูลระดับความยาก

ค่าของด่านถูกจัดเก็บใน LevelConfig ซึ่งเป็น ScriptableObject ทำให้ปรับสมดุลเกมได้โดยไม่เปลี่ยนตรรกะหลัก ตารางต่อไปแสดงค่าหลักที่ใช้ในเกมเวอร์ชันปัจจุบัน

พารามิเตอร์	Level 1	Level 2	Level 3
ระดับความยาก	Easy	Normal	Nightmare
กุญแจที่ต้องใช้	5	5	5
พลังชีวิตผู้เล่น	100	100	100
ความเร็วเดิน	4.5	4.5	4.5
ความเร็ววิ่ง	5.7	5.7	5.7
แบตเตอรี่ไฟฉาย	80	60	45
ความเร็วลาดตระเวน	2.0	1.3	2.8
ความเร็วไล่ล่า	4.0	3.0	7.2
ระยะมองเห็น	6.0	4.55	0
ระยะได้ยิน	4.0	3.25	10.0
พลังโจมตี	25	50	35

Level 3 ใช้ชื่อ Echoes of the Blind และกำหนดระยะมองเห็นของศัตรูเป็นศูนย์ แต่เพิ่มระยะได้ยินเป็น 10 หน่วย จึงเน้นการหลบเลี่ยงด้วยการควบคุมเสียงมากกว่าการหลบแนวสายตา

3.4 การออกแบบระบบ

3.4.1 การออกแบบข้อมูลนำเข้า

ข้อมูลนำเข้าประกอบด้วยคำสั่งจากแป้นพิมพ์ ตำแหน่งและทิศทางของผู้เล่น สถานะของ Player FSM ระยะห่างจากศัตรู สถานะไฟฉาย เหตุการณ์เสียง จำนวนกุญแจ ไอเท็มที่เก็บได้ และข้อมูล LevelConfig ของด่านปัจจุบัน ข้อมูลเหล่านี้ถูกส่งไปยังโมดูลที่รับผิดชอบเพื่อประมวลผลตามเงื่อนไข

3.4.2 การออกแบบกระบวนการทำงาน

1) GameManager โหลด LevelConfig และฉากของด่านปัจจุบัน

2) PlayerController รับคำสั่งและตรวจสอบ PlayerState ก่อนดำเนินการ
3) ระบบคำนวณการเคลื่อนที่ Stamina การกระโดด การไต่บันได และการโต้ตอบ
4) FootstepSystem หรือวัตถุในฉากส่งเหตุการณ์เสียงไปยัง KillerAI
5) KillerAI ประเมินการมองเห็น เสียง และระดับความสงสัยก่อนเปลี่ยนสถานะ
6) InventoryManager ตรวจสอบไอเท็มและจำนวนกุญแจ
7) ExitDoor ตรวจสอบเงื่อนไขและความคืบหน้าการปลดล็อก
8) UIManager และระบบเสียงอัปเดตผลลัพธ์ให้ผู้เล่นรับรู้
9) GameManager เปลี่ยนด่านหรือแสดงผลการจบเกมเมื่อเงื่อนไขสำเร็จ

3.4.3 การออกแบบเอาต์พุต

หน้าจอเมนูหลักประกอบด้วยปุ่ม Start สำหรับเริ่มเกม Options สำหรับเปิดหน้าตั้งค่าเสียง และ Quit สำหรับออกจากเกม โดยใช้โทนสีเข้มให้สอดคล้องกับบรรยากาศสยองขวัญ

[แทรกภาพ: Main Menu ที่มีปุ่ม Start Options และ Quit]

ภาพที่ 3-1 หน้าจอเมนูหลัก

หน้าจอตั้งค่าประกอบด้วย Slider สำหรับ Master Volume, SFX Volume และ Music Volume โดยค่าที่ปรับถูกเก็บด้วย PlayerPrefs เพื่อเรียกใช้ในการเปิดเกมครั้งต่อไป

[แทรกภาพ: Options Menu และ Slider เสียงทั้งสามรายการ]

ภาพที่ 3-2 หน้าจอตั้งค่าเสียง

HUD แสดงชื่อด่าน ระดับความยาก จำนวนกุญแจ พลังชีวิต แบตเตอรี่ไฟฉาย Stamina จำนวนก้อนหิน และข้อความโต้ตอบ รวมทั้ง HIDDEN และ DANGER ตามสถานการณ์

[แทรกภาพ: HUD ขณะเล่นที่เห็นข้อมูลหลักครบ]

ภาพที่ 3-3 หน้าจอระหว่างเล่น

เมื่อกด Left Shift ระหว่างเคลื่อนที่ ผู้เล่นจะวิ่งเร็วขึ้นและ Stamina ลดลง แถบจะเปลี่ยนสีเมื่อเหลือพลังงานต่ำและฟื้นกลับหลังหยุดใช้

[แทรกภาพ: ผู้เล่นกำลังวิ่งพร้อมแถบ Stamina]

ภาพที่ 3-4 การวิ่งและระบบ Stamina

เมื่อเข้าใกล้วัตถุที่โต้ตอบได้ ระบบแสดง Prompt เพื่อแจ้งคำสั่ง ผู้เล่นกด E เพื่อเก็บไอเท็มหรือใช้งานวัตถุ

[แทรกภาพ: Interaction Prompt ขณะเข้าใกล้กุญแจ]

ภาพที่ 3-5 การโต้ตอบกับไอเท็ม

ผู้เล่นกด C เพื่อเปิดและปิดคลังไอเท็ม
หน้าต่างแสดงช่องไอเท็มที่เก็บได้และรองรับการเลือกใช้ไอเท็มบางประเภท

[แทรกภาพ: Inventory จริงจากเกม]

ภาพที่ 3-6 หน้าจอคลังไอเท็ม

เมื่อผู้เล่นอยู่ใกล้จุดซ่อนและกด H ระบบเปลี่ยนสถานะเป็น Hidden พร้อมแสดง HIDDEN บนหน้าจอ

[แทรกภาพ: ผู้เล่นในจุดซ่อนและข้อความ HIDDEN]

ภาพที่ 3-7 ระบบซ่อนตัว

ผู้เล่นกด F เพื่อเปิดหรือปิดไฟฉาย แบตเตอรี่ลดลงระหว่างใช้งาน
และการเปิดไฟฉายเพิ่มระยะตรวจพบด้วยสายตาของศัตรู

[แทรกภาพ: แสงไฟฉายและแถบ Battery]

ภาพที่ 3-8 ระบบไฟฉายและแบตเตอรี่

ผู้เล่นขว้างก้อนหินเพื่อสร้างเสียงจากตำแหน่งอื่น ศัตรูรับเสียงประเภท Rock
และเดินไปตรวจสอบ หากใช้ซ้ำบริเวณเดิมหลายครั้งศัตรูจะลดความเชื่อถือของเสียงล่อ

[แทรกภาพ: ก้อนหินตกและศัตรูเดินไปตรวจสอบ]

ภาพที่ 3-9 การใช้ก้อนหินล่อศัตรู

เมื่อศัตรูตรวจพบผู้เล่น ระบบเปลี่ยนเข้าสู่ Chase HUD แสดง DANGER
และระบบเสียงปรับเข้าสู่บรรยากาศการไล่ล่า

[แทรกภาพ: Killer ไล่ล่าผู้เล่นและ HUD แสดง DANGER]

ภาพที่ 3-10 การตรวจพบและไล่ล่าผู้เล่น

เมื่อศัตรูอยู่ในระยะโจมตี ระบบลดพลังชีวิตของผู้เล่น หากพลังชีวิตเป็นศูนย์ Player FSM เปลี่ยนเป็น Dead และแสดงหน้าจอ Game Over

[แทรกภาพ: ผู้เล่นได้รับความเสียหายและหน้าจอ Game Over]

ภาพที่ 3-11 การได้รับความเสียหายและ Game Over

หลังเก็บกุญแจครบ ผู้เล่นต้องกด E ค้างที่ประตูจนความคืบหน้าครบ 100 เปอร์เซ็นต์
การปลดล็อกจะถูกยกเลิกเมื่อปล่อยปุ่ม ออกจากระยะ ถูกโจมตี หรือมีศัตรูอยู่ใกล้

[แทรกภาพ: ข้อความ Hold E to unlock และเปอร์เซ็นต์]

ภาพที่ 3-12 การปลดล็อกประตูทางออก

เมื่อปลดล็อกประตูสำเร็จ ระบบแสดง Level Complete และโหลดด่านถัดไป หากผ่านด่านสุดท้ายระบบแสดง Game Won

[แทรกภาพ: หน้าจอ Level Complete และ Game Won]

ภาพที่ 3-13 หน้าจอผ่านด่านและจบเกม

Level 2 มีสถานะคำสาป Safe, Exposed, Warning, Critical, Petrified และ Recovering โดย HUD แสดงแถบคำสาป ภาพบดบัง และข้อความเตือนตามระดับอันตราย

[แทรกภาพ: Curse HUD ของ Level 2]

ภาพที่ 3-14 ระบบคำสาปใน Level 2

Level 3 ออกแบบเป็นป่ามืดโดยใช้หมอก ต้นสนระยะไกล ต้นไม้และพุ่มไม้ชั้นกลาง รวมถึงพื้นหญ้าและเส้นทางต่างระดับเพื่อสร้างความลึกของฉาก

[แทรกภาพ: ภาพรวมฉากป่าของ Level 3]

ภาพที่ 3-15 สภาพแวดล้อมป่าของ Level 3

กับดักกระจกทำงานเมื่อผู้เล่นเดินชน โดยเล่นเสียงแตก เปลี่ยนลักษณะวัตถุ และส่งเสียงประเภท Glass ไปยัง KillerAI

[แทรกภาพ: ก่อนและหลังเหยียบกับดักกระจก]

ภาพที่ 3-16 กับดักเสียงใน Level 3

เมื่อศัตรูอยู่ในระยะเตือน เกมเล่นเสียงหัวใจและแสดงขอบจอสีแดง ความถี่และความเข้มเพิ่มขึ้นตามระดับอันตราย

[แทรกภาพ: ขอบจอสีแดงขณะ Killer อยู่ใกล้]

ภาพที่ 3-17 ระบบแจ้งเตือนเมื่อศัตรูอยู่ใกล้

3.5 การพัฒนาระบบ

3.5.1 สถาปัตยกรรมของระบบ

ระบบประกอบด้วย PlayerController, PlayerHealth, FlashlightController, HidingSystem, FootstepSystem, KillerAI, InventoryManager, ExitDoor, GameManager, UIManager, AudioManager และ TensionDirector โดย PlayerController รับข้อมูลจากผู้เล่นและส่งเหตุการณ์ให้ระบบที่เกี่ยวข้อง FootstepSystem ส่งเหตุการณ์เสียงให้ KillerAI

ขณะที่ InventoryManager ส่งจำนวนกุญแจให้ UIManager และ ExitDoor ส่วน GameManager ควบคุมสถานะเกมและการเปลี่ยนฉาก

3.5.2 การพัฒนา Player FSM

Player FSM ใช้สถานะ Normal, Hidden, Frozen, Hurt, UsingItem, Transitioning และ Dead เพื่อควบคุมการรับคำสั่ง Normal เป็นสถานะที่อนุญาตการควบคุมทั่วไป Hidden ใช้เมื่ออยู่ในจุดซ่อน Frozen และ Hurt ใช้หยุดหรือจำกัดการควบคุมชั่วคราว UsingItem ป้องกันคำสั่งที่ขัดแย้งระหว่างใช้ไอเท็ม Transitioning ใช้ระหว่างเปลี่ยนฉาก และ Dead ใช้เมื่อพลังชีวิตหมด

3.5.3 การพัฒนา Enemy FSM

Enemy FSM รับข้อมูลจากระบบการมองเห็นและการได้ยิน จากนั้นประเมินความสงสัยและเลือกสถานะ เสียง Glass และ Sprint มีความสำคัญมากกว่าเสียง Footstep ทั่วไป ศัตรูเก็บตำแหน่งล่าสุดไว้ชั่วคราวเพื่อใช้ตรวจสอบและค้นหา หากไม่พบเป้าหมายจะกลับไปยังเส้นทางลาดตระเวน

3.5.4 การพัฒนาระบบความตึงเครียด

TensionDirector รวมสถานะของศัตรูทุกตัวเป็น Calm, Suspicious, Hunted, Escaped และ Cooldown ข้อมูลนี้ใช้ควบคุมระดับความตึงเครียด ดนตรีไล่ล่า เสียงหัวใจ ภาพเตือน และตัวคูณความเร็วของศัตรูระหว่างการไล่ล่า

3.5.5 การพัฒนาระบบไอเท็มและประตูทางออก

InventoryManager เก็บไอเท็มใน List และส่งเหตุการณ์เมื่อจำนวนกุญแจเปลี่ยน ExitDoor มีสถานะ Locked, Ready, Unlocking, Open, Transitioning และ Completed การใช้ FSM กับประตูช่วยควบคุมการปลดล็อกแบบกดค้างและรองรับการยกเลิกเมื่อสถานการณ์ไม่ปลอดภัย

3.5.6 การพัฒนาด่าน Level 3

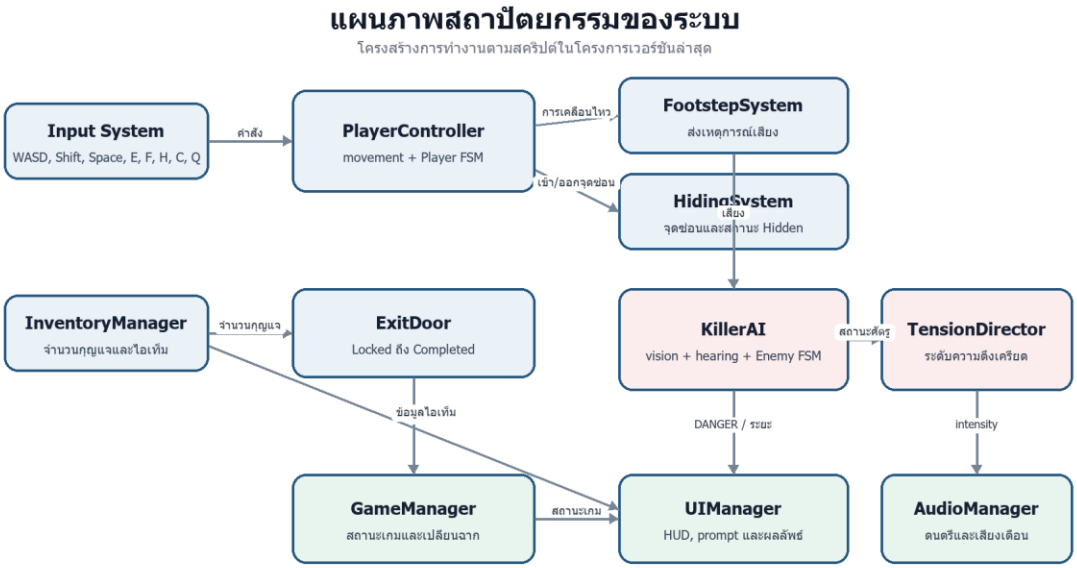
Level 3 ใช้ Tilemap และ Sprite ที่มีอยู่ในโครงการจัดวางเป็นป่า โดยแบ่งเป็นชั้นหมอกและต้นไม้ระยะไกล ชั้นต้นไม้และพุ่มไม้ระยะกลาง และชั้นพื้นสำหรับการเล่น ระบบ Level3RandomEvents ค้นหาพื้นที่เดินได้และกระจายกุญแจกับกับดักระหว่างเส้นทางชั้นล่างและชั้นบน เพื่อลดการรวมตัวของวัตถุในตำแหน่งเดียว

3.5.7 การจัดการเงื่อนไขพิเศษ

ระบบป้องกันการกระโดดซ้ำกลางอากาศ รองรับการกระโดดออกจากบันได ล็อกการเคลื่อนไหวขณะซ่อน ใช้ไอเท็ม เปลี่ยนฉาก หรือเสียชีวิต

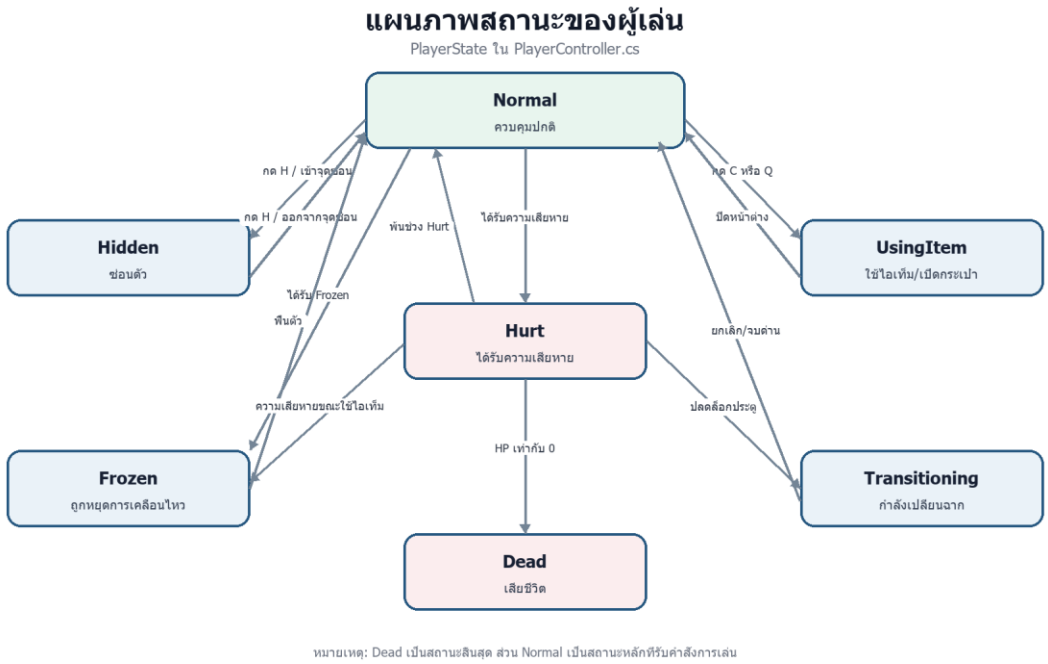
ลบเสียงที่หมดอายุจากหน่วยความจำของศัตรู ลดประสิทธิภาพการล่อด้วยก้อนหินซ้ำ และยกเลิกการปลดล็อกประตูเมื่อผู้เล่นอยู่ในอันตราย

แผนภาพแสดงการรับข้อมูลจาก Input System ผ่าน PlayerController การส่งเหตุการณ์ไปยังระบบไอเท็ม ระบบเสียง KillerAI UI และ GameManager



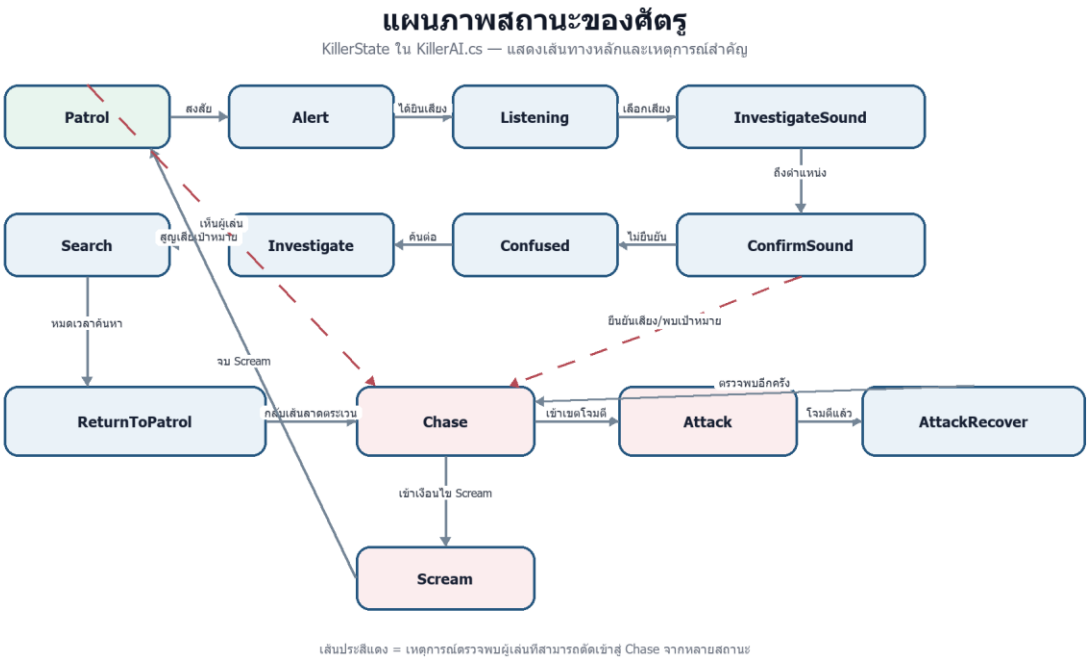
ภาพที่ 3-18 แผนภาพสถาปัตยกรรมของระบบ

แผนภาพแสดงสถานะของผู้เล่นและเหตุการณ์สำคัญที่ทำให้เปลี่ยนระหว่าง Normal, Hidden, Frozen, Hurt, UsingItem, Transitioning และ Dead



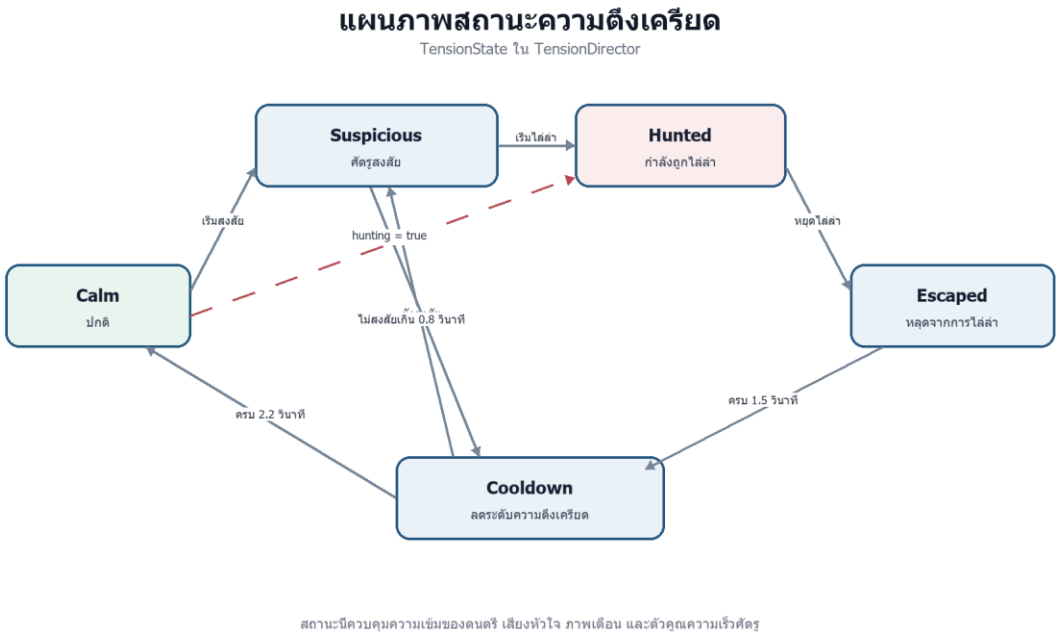
ภาพที่ 3-19 แผนภาพ Player FSM

แผนภาพแสดงกระบวนการตั้งแต่ Patrol การรับรู้เสียงหรือการมองเห็น การตรวจสอบ การค้นหา การไล่ล่า การโจมตี และการกลับไปลาดตระเวน



ภาพที่ 3-20 แผนภาพ Enemy FSM

แผนภาพแสดงสถานะ Calm, Suspicious, Hunted, Escaped และ Cooldown ซึ่งควบคุมเสียงและภาพเตือนผู้เล่น



ภาพที่ 3-21 แผนภาพสถานะความตึงเครียด

3.6 การทดสอบระบบ

การทดสอบใช้กรณีทดสอบหรือ Test Case กำหนดข้อมูลนำเข้า ขั้นตอน และผลลัพธ์ที่คาดหวังอย่างชัดเจน
โดยต้องบันทึกผลจริงจากการเล่นทดสอบก่อนระบุสถานะผ่านหรือไม่ผ่าน
ตารางในหัวข้อนี้จึงเว้นผลจริงไว้สำหรับกรอกหลังดำเนินการทดสอบ

3.6.1 การทดสอบระบบควบคุมพื้นฐาน

รหัส	รายการทดสอบ	ขั้นตอน	ผลที่คาดหวัง	ผลจริง/สถานะ
TC-01	การเดิน	กด A D หรือปุ่มลูกศร	ผู้เล่นเคลื่อนที่และหันถูกทิศ	รอบันทึกผล
TC-02	การวิ่ง	กด Shift พร้อมเคลื่อนที่	ความเร็วเพิ่ม Stamina ลด และเกิดเสียง Sprint	รอบันทึกผล
TC-03	การกระโดด	กด Space บนพื้นและกลางอากาศ	กระโดดได้ครั้งเดียวและไม่กระโดดซ้ำกลางอากาศ	รอบันทึกผล
TC-04	การไต่บันได	กดขึ้นหรือลงเมื่ออยู่บนบันได	ผู้เล่นไต่บันไดและกระโดดออกได้	รอบันทึกผล

รหัส	รายการทดสอบ	ขั้นตอน	ผลที่คาดหวัง	ผลจริง/สถานะ
TC-05	ไฟฉาย	กด F และเปิดทิ้งไว้	ไฟฉายสลับสถานะและแบตเตอรี่ลดลง	รอบันทึกผล

3.6.2 การทดสอบ Player FSM

รหัส	การเปลี่ยนสถานะ	เงื่อนไขทดสอบ	ผลที่คาดหวัง	ผลจริง/สถานะ
TC-FSM-P01	Normal > Hidden	เข้าใกล้จุดซ่อนและกด H	ผู้เล่นซ่อนและ HUD แสดง HIDDEN	รอบันทึกผล
TC-FSM-P02	Hidden > Normal	กด H เพื่อออกจากจุดซ่อน	กลับมาเคลื่อนที่ได้ตามปกติ	รอบันทึกผล
TC-FSM-P03	Normal > Hurt > Normal	ให้ศัตรูโจมตีโดย HP ยังเหลือ	HP ลดและกลับสู่ Normal หลังช่วง Hurt	รอบันทึกผล
TC-FSM-P04	Normal > Frozen > Normal	รับผล Frozen และทำเงื่อนไขหลุด	หยุดเคลื่อนที่และกลับมาควบคุมได้	รอบันทึกผล
TC-FSM-P05	Normal > UsingItem > Normal	เปิดและปิด Inventory	คำสั่งการเล่นถูกล็อกและคืนกลับถูกต้อง	รอบันทึกผล
TC-FSM-P06	Normal > Transitioning	ปลดล็อกประตูสำเร็จ	ล็อกการควบคุมระหว่างเปลี่ยนด่าน	รอบันทึกผล
TC-FSM-P07	Hurt > Dead	รับความเสียหายจน HP เป็นศูนย์	เข้าสู่ Dead และแสดง Game Over	รอบันทึกผล

3.6.3 การทดสอบ Enemy FSM

รหัส	การเปลี่ยนสถานะ	เงื่อนไขทดสอบ	ผลที่คาดหวัง	ผลจริง/สถานะ
TC-FSM-E01	Patrol > Listening	สร้างเสียงในระยะได้ยิน	ศัตรูหยุดลาดตระเวนและฟังเสียง	รอบันทึกผล
TC-FSM-E02	Listening > InvestigateSound	รอจนศัตรูเลือกเสียง	ศัตรูเคลื่อนไปยังต้นกำเนิดเสียง	รอบันทึกผล
TC-FSM-E03	InvestigateSound > ConfirmSound	ศัตรูถึงตำแหน่งเสียง	ศัตรูตรวจสอบบริเวณดังกล่าว	รอบันทึกผล
TC-FSM-E04	Patrol/Investigate > Chase	ให้ศัตรูตรวจพบผู้เล่น	ศัตรูไล่ล่าและ HUD แสดง DANGER	รอบันทึกผล
TC-FSM-E05	Chase > Attack	ผู้เล่นอยู่ในระยะโจมตี	ศัตรูโจมตีและลด HP	รอบันทึกผล
TC-FSM-E06	Attack > AttackRecover	รอหลังการโจมตี	ศัตรูพักตามช่วง cooldown	รอบันทึกผล
TC-FSM-E07	Chase > Investigate/Search	หลบออกจากการตรวจจับ	ศัตรูตรวจตำแหน่งล่าสุดและค้นหา	รอบันทึกผล
TC-FSM-E08	Search > ReturnToPatrol	รอจนหมดเวลาค้นหา	ศัตรูกลับเส้นทางลาดตระเวน	รอบันทึกผล
TC-FSM-E09	Listening > Confused	ขว้างก้อนหินซ้ำบริเวณเดิม	ศัตรูเข้าสู่ Confused ตามเงื่อนไข	รอบันทึกผล
TC-FSM-E10	สถานะทั่วไป > Scream	ทำให้ศัตรูที่ตั้งค่า Scream เข้าเงื่อนไข	ผู้เล่นถูก Stun หรือ Frozen	รอบันทึกผล

3.6.4 การทดสอบระบบเกมและ Level 3

รหัส	รายการทดสอบ	ขั้นตอน	ผลที่คาดหวัง	ผลจริง/สถานะ
TC-06	เก็บกุญแจ	เก็บกุญแจหนึ่งชิ้น	จำนวนกุญแจและ Inventory อัปเดต	รอบันทึกผล
TC-07	ประตูยังล็อก	ใช้ประตูก่อนกุญแจครบ	แสดงจำนวนกุญแจที่ยังขาด	รอบันทึกผล
TC-08	ปลดล็อกประตู	เก็บครบและกด E ค้าง	ความคืบหน้าครบและผ่านด่าน	รอบันทึกผล
TC-09	ยกเลิกการปลดล็อก	ปล่อย E เดินออก หรือถูกโจมตี	ความคืบหน้าถูกยกเลิก	รอบันทึกผล
TC-10	กับดักกระจก	เหยียบกับดักใน Level 3	เกิดเสียง Glass และศัตรูตรวจสอบ	รอบันทึกผล
TC-11	การสุ่มวัตถุ	เข้า Level 3 หลายรอบ	กุญแจและกับดักกระจายตามจุดที่เดินได้	รอบันทึกผล
TC-12	เสียงหัวใจ	เข้าใกล้ศัตรู	ความถี่เสียงและขอบจอแดงเพิ่มตามอันตราย	รอบันทึกผล
TC-13	จบเกม	ผ่าน Level 3	ระบบแสดง Game Won	รอบันทึกผล

3.6.5 การทดสอบโดยผู้ใช้งาน

การทดสอบโดยผู้ใช้งานมีวัตถุประสงค์เพื่อประเมินความถูกต้อง ความเสถียร ความเข้าใจง่ายของการควบคุม ความชัดเจนของ UI ความเหมาะสมของระดับความยาก และบรรยากาศของเกม ผู้ทดสอบให้คะแนนด้วยมาตราส่วน 5 ระดับ ได้แก่ 5 ดีมาก 4 ดี 3 ปานกลาง 2 น้อย และ 1 น้อยมาก

ผลการประเมินจะวิเคราะห์ด้วยค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐาน ทั้งนี้ต้องระบุจำนวนผู้ทดสอบ คุณสมบัติของกลุ่มตัวอย่าง วันที่หรือช่วงเวลาทดสอบ และผลคะแนนจริงก่อนนำเสนอข้อสรุป

3.7 การบำรุงรักษาระบบ

3.7.1 การบำรุงรักษาเพื่อแก้ไขข้อผิดพลาด

ตรวจสอบและแก้ไขปัญหาการเปลี่ยนสถานะของผู้เล่นและศัตรู การติดขอบพื้น การเดินบนสะพาน การตรวจจับจุดซ่อน การปลดล็อกประตู และการติดตามกล้องในพื้นที่ต่างระดับ

3.7.2 การบำรุงรักษาเพื่อปรับตัว

ปรับ Input System ความละเอียด อัตราส่วนภาพ ขอบเขตกล้อง และค่าประสิทธิภาพ เพื่อให้รองรับอุปกรณ์หรือสภาพแวดล้อมที่แตกต่างกัน

3.7.3 การบำรุงรักษาเพื่อพัฒนาเพิ่มเติม

เพิ่มชนิดศัตรู ไอเท็ม กับดัก ด่าน และสถานะ FSM ใหม่ได้ผ่านโครงสร้างโมดูล พร้อมปรับความยากของด่านผ่าน LevelConfig โดยไม่เปลี่ยนตรรกะหลัก

3.7.4 การบำรุงรักษาเชิงป้องกัน

ทดสอบทุกด่านหลังแก้ไขระบบส่วนกลาง ตรวจสอบ Unity Console ไม่ให้มีข้อผิดพลาด สำรอง Scene และ Prefab ก่อนแก้ไข และอัปเดตแผนภาพ ตาราง
รวมถึงภาพหน้าจอในรายงานให้ตรงกับเกมเวอร์ชันล่าสุด